

Nesne Yönelimli Yaklaşımla K-En Yakın Komşu ve Karar Ağacı Algoritmalarının Karşılaştırmalı Analizi

Burak Kılıcı
Bilgisayar Mühendisliği
Kocaeli Üniversitesi
Öğrenci No: 230202111

Ömer Faruk Teke
Bilgisayar Mühendisliği
Kocaeli Üniversitesi
Öğrenci No: 210202045

Özet—Bu çalışmada, Kocaeli ilinde Mart 2017 döneminde gerçekleşmiş 5094 satışlık gerçek bir perakende veri kümesi üzerinde, müşterilerin sosyo-ekonomik profilinden yola çıkarak ürün kategorisini tahmin eden iki denetimli öğrenme algoritması Java programlama dilinde, herhangi bir makine öğrenmesi kütüphanesi kullanılmadan sıfırdan geliştirilmiştir. K-En Yakın Komşu (KNN) ve Karar Ağacı (DT) algoritmaları, Öklid mesafesi ve Information Gain gibi temel matematiksel formüller ile kodlanmış; veri yükleme, ön işleme, sınıflandırma ve değerlendirme adımları Nesne Yönelimli Programlama (NYP) prensiplerine uygun modüler bir mimari ile inşa edilmiştir. Deneysel sonuçlara göre Karar Ağacı algoritması (maksimum derinlik 12) %86,27 doğruluk oranıyla KNN'in ($k = 3$, %77,41) önüne geçmiştir; ayrıca tahmin süresinde yaklaşık 130 kat daha hızlı çalışmıştır. Bulgular, kural tabanlı algoritmaların kategorik özellik yoğunluklu veri kümelerinde mesafe tabanlı algoritmalarla kıyasla hem doğruluk hem performans açısından avantajlı olduğunu göstermektedir.

Index Terms—K-En Yakın Komşu, Karar Ağacı, Nesne Yönelimli Programlama, Sınıflandırma, Java, Information Gain, Entropi

I. GİRİŞ

Perakende sektöründe müşteri davranışlarını önceden tahmin etmek; stok yönetiminden hedefli pazarlamaya, raf yerleşim kararlarından müşteri segmentasyonuna kadar pek çok stratejik kararın temelini oluşturur. Denetimli öğrenme (supervised learning) kapsamındaki sınıflandırma algoritmaları, geçmiş satın alma verisinden yararlanarak yeni müşterilerin hangi ürün kategorisine yöneleceğini öneri düzeyinde tahmin edebilir. Bu algoritmaların pek çoğu hazır kütüphaneler aracılığıyla kullanıma sunulsa da, algoritmaların iç işleyişinin kavranması ve özelleştirilebilir bir sistem tasarlanması açısından sıfırdan geliştirilmesi önemli bir eğitsel değer taşır.

Bu çalışmanın amacı, K-En Yakın Komşu (KNN) ve Karar Ağacı (Decision Tree – DT) algoritmalarının tahmin başarısını, çalışma süresini ve sınıflar arası hata dağılımını Kocaeli merkezli gerçek bir perakende veri kümesi üzerinde karşılaştırmalı olarak analiz etmektir. Projede iki temel kısıt vardır: Birincisi, Weka veya Scikit-learn gibi hazır makine öğrenmesi kütüphanelerinin kullanılmaması; ikincisi, tüm mimarinin Nesne Yönelimli Programlama (NYP) prensipleri (Kapsülleme, Soyutlama, Kalıtım, Çok Biçimlilik) üzerine kurulmasıdır.

KNN, “benzer profile sahip insanlar benzer ürünlere yönelir” sezgisini kullanan, mesafe tabanlı ve tembel öğrenen (lazy learning) bir algoritmadır [2]. Eğitim aşamasında hiçbir hesaplama yapmaz; tüm maliyeti tahmin aşamasına kaydırır. **Karar Ağacı** ise veriyi en ayırt edici özelliklere göre özyineli olarak dallandıran, insan tarafından okunabilir kurallar hiyerarşisi üreten bir algoritmadır [3]. Eğitim aşamasında daha maliyetlidir, fakat tahmin aşamasında ağacın yüksekliğine bağlı olarak logaritmik bir karmaşıklıkla çalışır. Her iki yaklaşımın avantaj ve dezavantajları bu çalışmada somut ölçümlerle ortaya konmuştur.

Raporun geri kalan bölümleri şu şekilde yapılandırılmıştır: Bölüm II, kullanılan veri kümesini ve uygulanan ön işleme adımlarını detaylandırmaktadır. Bölüm III, KNN ve Karar Ağacı algoritmalarının çalışma mantığını ve matematiksel alt-yapısını açıklamaktadır. Bölüm IV, projenin NYP mimarisini ve uygulanan prensipleri anlatmaktadır. Bölüm V ve VI deneysel sonuçları ve yorumlarını sunmakta; Bölüm VII ise elde edilen bulguları ve gelecek çalışmaları özetlemektedir.

II. VERİ SETİ

A. Kaynak ve Yapı

Çalışmada kullanılan veri seti, “Turkish Market Sales Data” veri kümesinden Kocaeli iline filtrelenmiş bir alt kümedir. Mart 2017’ye ait 5094 satışlık bu ham veri, 18 sütun içermektedir. Her satır bir satış işlemini (transaction) temsil eder. Bağımlı değişken (tahmin edilecek hedef), CATEGORY_NAME1 sütunudur ve 11 benzersiz ana ürün kategorisi (GIDA, MEYVE SEBZE, SÜT KAHVALTILIK, İÇECEK, DETERJAN TEMİZLİK, ET TAVUK, KOZMETİK, KAĞIT, SİGARA, EV, BEBEK) içerir. Kategorilerin dağılımı dengesiz olup GIDA kategorisi toplam verinin yaklaşık %33’ünü, BEBEK kategorisi ise yalnızca %0,9’unu oluşturmaktadır. Bu dengesizlik, modelin başarısı üzerinde belirleyici bir etkiye sahip olup sonuç bölümünde ayrıntılı ele alınacaktır.

B. Özellik Seçimi ve Gerekçendirme

Ham verideki 18 sütundan 6 tanesi model girdisi olarak seçilmiştir: CLIENTCODE (müşteri kimliği), GENDER (cinsiyet), LINENETTOTAL (toplam tutar), BRANDCODE ve BRAND

(marka kodu/adı), CATEGORY_NAME1 (hedef). Geri kalan 12 sütun aşağıdaki gerekçelerle dışlanmıştır:

- **Veri sızıntısı (data leakage) riski:** ITEMNAME, ITEMCODE, CATEGORY_NAME2 ve CATEGORY_NAME3 sütunları, hedef değişkenle doğrudan özdeş ilişki taşır. Örneğin “PINAR SÜT 1L” ürün adı verildiğinde kategori zaten “SÜT KAHVALTILIK” olmak zorundadır; bu tür özelliklerin modele verilmesi, gerçek bir tahmin değil eşleme görevi yarattığından dışlanmıştır.
- **Meta veri:** ID, FICHENO, STARTDATE, ENDDATE gibi sütunlar rastgele benzersiz değerler taşır; tahmin değeri yoktur.
- **Fazlalık bilgi:** AMOUNT ve PRICE sütunları matematiksel olarak LINENETTOTAL = AMOUNT × PRICE ilişkisinde olduğundan LINENETTOTAL üzerinden dolayı olarak zaten modele yansıtılmaktadır.

C. Veri Temizleme

Ham veri setinde eksik (null) değer içeren 469 satır tespit edilmiştir. Bu satırlar veri yükleme aşamasında VeriYukleyici sınıfı tarafından try-catch bloğu ile ayıklanmış, sayısal alanlardaki hatalı tip dönüşümleri NumberFormatException yakalanarak güvenli geçilmiştir. Temizleme sonrası 4625 geçerli kayıt elde edilmiştir. PDF’te istenildiği üzere bu aşamada hiçbir hazır kütüphaneye başvurulmamış, dosya okuma BufferedReader ile satır satır gerçekleştirilmiştir.

D. Sayısal Dönüştürme (Label Encoding)

KNN gibi mesafe tabanlı algoritmalar metinlerle işlem yapamadığından kategorik alanlar sayıya dönüştürülmüştür:

- **Cinsiyet:** Veri setinde “E” (erkek) ve “K” (kadın) olarak bulunan değerler sırasıyla 1 ve 0 olarak kodlanmıştır. (Not: Proje dokümanında örnek olarak verilen Male/Female ifadelerinin aksine, asıl veri setinde “E”/“K” değerleri bulunmaktadır; kodlamada gerçek veriye uyum sağlanmıştır.)
- **Marka kodu:** 266 benzersiz marka HashMap yapısı kullanılarak 0 ile 265 arasında benzersiz tam sayılara eşlenmiştir. Aynı marka kaydına her karşılaşıldığında aynı kod atanmıştır.

E. Normalizasyon

KNN’de Öklid mesafesinin adil hesaplanabilmesi için tüm sayısal özellikler **Min-Max** normalizasyonu ile [0, 1] aralığına çekilmiştir:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (1)$$

Bu adım olmadan, [0, 265] aralığındaki marka kodu, [0, 1] aralığındaki cinsiyet özelliğini mesafe hesabında ezmekte ve modelin yalnızca marka bilgisine yaslanmasına sebep olmaktadır. Tutar için gözlemlenen aralık [0, 100] olmuş, marka kodu için ise [0; 265] aralığı normalize edilmiştir.

F. Eğitim-Test Ayrımı

Temizlenmiş 4625 kayıt, Collections.shuffle fonksiyonu ile rastgele karıştırılmış (seed: 42, tekrarlanabilirlik için) ve PDF’te belirtildiği üzere %80 eğitim (3700 kayıt), %20 test (925 kayıt) olarak ayrılmıştır. Veri sızıntısını engellemek için eğitim ve test kayıtları tamamen ayrı tutulmuştur. Tohum değerinin (seed = 42) sabit tutulması, sonuçların farklı çalıştırmalar arasında tekrar edilebilmesini sağlamıştır.

III. YÖNTEM

A. K-En Yakın Komşu (KNN)

KNN, bir sorgu noktasına en yakın K eğitim örneğine bakar ve aralarındaki çoğunluk sınıfını tahmin olarak döner. Algoritmanın temel varsayımı, benzer özelliklere sahip noktaların benzer sınıflara ait olacağıdır. İki kayıt arasındaki mesafe **Öklid mesafesi** ile hesaplanır:

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2} \quad (2)$$

Bu çalışmada $n = 3$ boyutlu özellik vektörü kullanılmıştır: *cinsiyet kodu, normalize tutar, normalize marka kodu*.

Performans optimizasyonu: 3700 eğitim kaydının tamamı her tahminde sıralanmak yerine, en yakın K komşu bir PriorityQueue (max-heap) içinde tutulmuştur. Bu sayede her tahminin asimptotik karmaşıklığı $O(n \log n)$ yerine $O(n \log K)$ olmuş ve $K = 5$ için yaklaşık 185 kat hızlanma sağlanmıştır. K değeri için 1, 3, 5, 7, 9 kombinasyonları denenmiş; en iyi sonuç $K = 3$ ile elde edilmiştir.

Oylama: K komşu bulunduktan sonra her birinin ait olduğu kategori sayılır ve en çok oy alan kategori tahmin olarak döndürülür.

B. Karar Ağacı (Decision Tree)

Karar ağacı, veriyi özyineli olarak en bilgilendirici özelliğe göre ikiye bölen bir yapı kurar. Her iç düğüm bir “eşik sorusu” sorar, her yaprak düğüm ise bir kategori tahminini temsil eder. Bölme kriteri olarak **Information Gain** (bilgi kazancı) tercih edilmiştir:

$$IG(S, A) = H(S) - \sum_{v \in V(A)} \frac{|S_v|}{|S|} H(S_v) \quad (3)$$

Burada $H(S)$ veri kümesinin Shannon entropisini ifade eder [4]:

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i) \quad (4)$$

Entropinin düşük çıkması veri kümesinin “saf” (tek bir sınıf ağırlıklı) olduğunu; yüksek çıkması ise karışık olduğunu gösterir.

Eşik adayı seçimi: Bir özellik için tüm olası eşik değerlerini denemek hesaplama açısından ağırdır. Bu çalışmada, özelliğin sıralı benzersiz değerlerinden komşu değerlerin ortalamaları aday eşik olarak belirlenmiş, 20’den fazla aday oluşursa örnekleme ile 20’ye indirilmiştir.

Durma kriterleri: Özyineli bölme süreci şu durumlarda durur: maksimum derinliğe (maxDepth) ulaşıldığında, alt kümede minimum kayıt sayısından az kayıt kaldığında, alt kümedeki tüm kayıtlar aynı kategoriye ait olduğunda veya bölmeden elde edilecek bilgi kazancı sıfır veya negatif olduğunda.

Aşırı öğrenme kontrolü: Sınırsız derinlik, eğitim setini ezberlemeye yol açtığından maxDepth parametresi ile kontrol altına alınmıştır. Denemelerde 2, 5, 8 ve 12 değerleri karşılaştırılmıştır.

IV. NESNE YÖNELİMLİ PROGRAMLAMA MIMARISI

Projenin yazılım mimarisi, PDF’te zorunlu tutulan dört NYP prensibini karşılamak üzere modüler bir şekilde kurgulanmıştır. Toplam 12 sınıf, 6 pakete ayrılmıştır. Kodun tamamı herhangi bir dış kütüphane içermez; yalnızca Java’nın standart kütüphanesinden yararlanılmıştır.

A. Kapsülleme (Encapsulation)

Veri kümesindeki her bir satır, SatisKaydi sınıfı ile temsil edilir. Sınıfın tüm alanları private erişim belirleyicisine sahiptir; dışarıdan erişim yalnızca public getter metotlarıyla, atama ise Constructor üzerinden sağlanır. Ham alanlar için setter tanımlanmamıştır, zira veri bir kez oluşturulduktan sonra değişmemelidir. Encoding ve normalizasyon sonuçları kontrollü setter’larla işlenir.

B. Soyutlama (Abstraction)

Her iki algoritmanın aynı standartta çağrılabilmesi için ISiniflandirici arayüzü tanımlanmıştır:

```
public interface ISiniflandirici {
    void modeliEgit(
        List<SatisKaydi> egitimVerisi);
    String tahminEt(SatisKaydi gelenKayit);
    String algoritmaAdi();
}
```

Bu arayüz, algoritmaların iç işleyişini gizleyerek “sözleşme” davranışı sunar.

C. Kalıtım (Inheritance)

Her iki algoritmanın paylaştığı metotlar (başarı oranı hesaplama, eğitim-test ayırma, süre ölçme) TemelAlgoritma adlı bir abstract sınıfta toplanmıştır. KNNSiniflandirici ve KararAgaciSiniflandirici bu sınıftan extends ile kalıtım alır; sadece algoritmaya özel modeliEgit ve tahminEt metotlarını @Override ile yeniden tanımlar. Bu yapı kod tekrarını engellemiş, bakımı kolaylaştırmıştır.

D. Çok Biçimlilik (Polymorphism)

Çalışma zamanında algoritma seçimi, List<TemelAlgoritma> tipli bir listeye farklı alt sınıf nesnelerinin konulması ile sağlanmıştır:

```
List<TemelAlgoritma> algoritmalar =
    Arrays.asList(
        new KNNSiniflandirici(5),
        new KararAgaciSiniflandirici(8, 5)
    );
for (TemelAlgoritma algo : algoritmalar) {
```

```
    algo.egitZamanli(egitim);
    double basari =
        algo.basariOraniHesapla(test);
}
```

Aynı tahminEt çağrısının çalışma zamanında farklı davranışlar sergilemesi, NYP’nin kalbi olan dinamik bağlamanın (dynamic dispatch) uygulamalı örneğidir.

E. Sınıf Diyagramı Özeti

Uygulama aşağıdaki paket yapısıyla düzenlenmiştir: model (SatisKaydi), veri (VeriYukleyici, VeriHazirlayici), siniflandirici (ISiniflandirici, TemelAlgoritma, KNNSiniflandirici, KararAgaciSiniflandirici, Dugum), degerlendirme (PerformansOlcer), arayuz (AnaEkran, UygulamaDurumu ve dört sekme paneli).

Kullanıcı arayüzü **Java Swing** ile, dış kütüphane kullanılmadan geliştirilmiştir. Sekmeler sayesinde veri yükleme, model eğitimi, metrik gösterimi ve grafik çıktısı bölümlere ayrılmıştır. Grafikler (bar chart ve confusion matrix ısı haritası) Java2D kullanılarak paintComponent metodunun override edilmesi yoluyla sıfırdan çizilmiştir.

V. DENEYSEL SONUÇLAR

Tüm deneyler, Intel Core i5 8. nesil işlemcili bir Windows 11 sisteminde, JDK 17 üzerinde koşulmuştur. Her deney aynı tohum değeri (seed= 42) ile tekrarlanmış, sonuçlar tek bir çalıştırma üzerinden alınmıştır.

A. Algoritma Karşılaştırması

Tablo I, sekiz farklı model konfigürasyonunun doğruluğunu ve çalışma sürelerini karşılaştırmaktadır.

Tablo I: Algoritmaların doğruluk ve çalışma süresi karşılaştırması (925 test örneği).

Algoritma	Doğruluk (%)	Eğitim (ms)	Tahmin (ms)
KNN (k=3)	77,41	0	132
KNN (k=5)	74,59	0	124
KNN (k=7)	73,84	0	43
KNN (k=9)	71,68	0	50
DT (derinlik=2)	53,19	22	0
DT (derinlik=5)	67,57	126	1
DT (derinlik=8)	80,22	105	0
DT (derinlik=12)	86,27	111	1

B. Kategori Bazında Başarı Analizi

En başarılı model olan Karar Ağacı (derinlik=12) için her kategorinin ayrı ayrı doğruluk oranı Tablo II ile verilmiştir.

Tablo II: Karar Ağacı (derinlik=12) için kategori bazında doğruluk.

Kategori	Doğru	Toplam	Başarı (%)
MEYVE SEBZE	136	136	100,00
ET TAVUK	43	46	93,48
SİGARA	25	27	92,59
GIDA	333	360	92,50
SÜT KAHVALTILIK	113	126	89,68
İÇECEK	85	111	76,58
KAĞIT	19	30	63,33
KOZMETİK	13	22	59,09
DETERJAN TEMİZLİK	21	37	56,76
EV	8	16	50,00
BEBEK	2	14	14,29

C. Görsel Sonuçlar

Şekil 1 kategori bazında başarı oranlarını, Şekil 2 ise karışıklık matrisini (confusion matrix) görselleştirmektedir. Bar chart'ta renk kodlaması kullanılmış; %80 ve üzeri yeşil, %60–80 sarı, %40–60 turuncu, %40 altı kırmızı ile işaretlenmiştir. Bu renk geçişi modelin güçlü ve zayıf yönlerini bir bakışta göstermektedir. Karışıklık matrisinin köşegeninde yoğunlaşma modelin başarısını doğrulamakta; köşegen dışı koyu hücreler ise kategoriler arası karışıklıkları ortaya koymaktadır.

VI. DEĞERLENDİRME

A. Doğruluk Açısından

Karar Ağacı, en iyi KNN konfigürasyonunu (%77,41) yaklaşık 9 puan farkla geçerek %86,27 doğruluk elde etmiştir. Bu sonucun temel nedeni, veri setinin en bilgilendirici özelliği olan **marka kodunun kategorik doğasıdır**: Karar Ağacı marka üzerinden if-else eşlemeleri ile doğrudan kurallar üretirken, KNN aynı bilgiyi Öklid mesafesinin bir koordinatı olarak dolaylı yoldan kullanmak zorundadır. Kategorik verinin sürekli bir mesafe uzayına sıkıştırılması bilgi kaybına yol açmış ve KNN'in performansını sınırlamıştır.

B. Çalışma Süresi Açısından

Tahmin süresinde Karar Ağacı, KNN'den **yaklaşık 130 kat daha hızlıdır** (1 ms'ye karşılık 132 ms). Bu, algoritma tasarımının doğal bir sonucudur: KNN her tahminde 3700 eğitim noktasına mesafe hesabı yapar ($O(n)$); Karar Ağacı ise ağacın yüksekliği kadar karşılaştırma yapar ($O(\log n)$). Öte yandan KNN'in eğitim aşaması (0 ms) Karar Ağacı'nın eğitim aşamasından (111 ms) çok daha hızlıdır. Algoritma seçimi bu sebeple uygulamanın eğitim/tahmin frekansına göre yapılmalıdır.

C. Sınıf Dengesizliğinin Etkisi

Veri seti aşırı dengesizdir: GIDA (1681 kayıt) ile BEBEK (40 kayıt) arasındaki oran yaklaşık 42:1'dir. Bu dengesizlik, Karar Ağacı'nın BEBEK sınıfındaki başarısını %14,29'a düşürmüş, buna karşılık MEYVE SEBZE kategorisini %100 doğrulukla tahmin etmesine yol açmıştır. Karışıklık matrisi incelendiğinde, emin olmadığı her durumda modelin ağırlıklı çoğunluk sınıfı olan GIDA'ya yöneldiği görülmektedir. Bu durum "çoğunluk sınıf yanlılığı" (majority class bias) olarak

bilinir ve literatürde SMOTE gibi tekniklerle çözülmektedir [7].

D. Derinlik Parametresinin Etkisi

Karar Ağacı deneylerinde maksimum derinliğin öğrenme kabiliyeti üzerindeki etkisi net biçimde gözlemlenmiştir. Derinlik 2 iken model yalnızca %53,19 ile **yetersiz öğrenme (underfitting)** sergilerken, derinlik 12'ye çıkarıldığında %86,27'ye ulaşmıştır. Derinlik 2 olan modelin karışıklık matrisi incelendiğinde, modelin tüm tahminlerini yalnızca iki kategoriye (GIDA ve MEYVE SEBZE) dağıttığı görülmüştür. Bununla birlikte aşırı büyümüş ağaçlarda **aşırı öğrenme (overfitting)** riski artmaktadır.

VII. SONUÇ

Bu çalışmada KNN ve Karar Ağacı algoritmaları, herhangi bir makine öğrenmesi kütüphanesi kullanılmadan, yalnızca Java'nın standart kütüphanesi ile sıfırdan geliştirilmiş; Ko-caeli'ye ait 4625 satırlık gerçek perakende verisi üzerinde karşılaştırılmıştır. Elde edilen bulgular şu şekildedir: (i) Karar Ağacı hem doğruluk hem tahmin hızında KNN'e üstünlük kurmuş, (ii) KNN hızlı eğitim ve basit mimari avantajlarını korumuş, (iii) sınıf dengesizliği her iki algoritmanın da az örneklenmiş kategorilerdeki performansını sınırlamıştır.

Nesne Yönelimli Programlama çerçevesinde kurulan mimari sayesinde yeni bir algoritmanın eklenmesi yalnızca TemelAlgoritma sınıfından kalıtım alan yeni bir sınıf yazmayı gerektirmektedir. Bu genişletilebilirlik, OOP'nin bakım ve sürdürülebilirlik kazanımlarını somutlaştırmıştır.

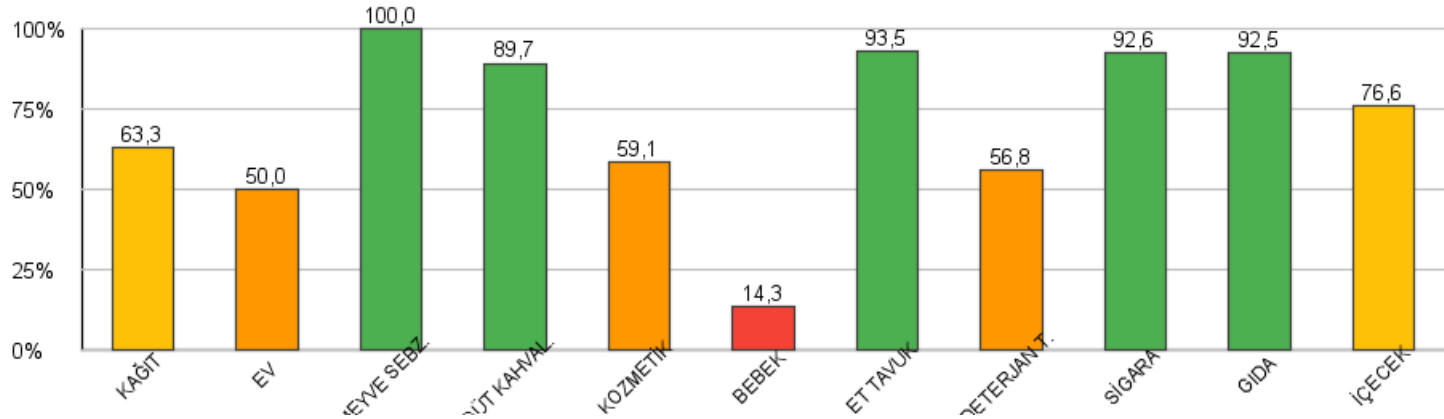
Gelecek çalışmalar için üç yön önerilmektedir: SMOTE gibi sınıf dengesizliği ile başa çıkma teknikleri, Random Forest gibi ensemble yöntemler ve yeni özellik mühendisliği (işlem saati, haftanın günü).

Karşılaşılan zorluklar arasında Türkçe karakter kodlaması, xlsx formatının saf Java ile okunamaması ve veri kümesi dengesizliği sayılabilir.

KAYNAKLAR

- [1] T. M. Mitchell, *Machine Learning*, 1st ed. New York: McGraw-Hill, 1997.
- [2] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967.
- [3] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
- [4] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948.
- [5] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Morgan Kaufmann, 2011.
- [6] Oracle Corporation, "Java SE 17 API Documentation," [Çevrimiçi]. Erişim: <https://docs.oracle.com/en/java/javase/17/docs/api/>
- [7] N. V. Chawla et al., "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

Kategori Bazında Başarı (%)



Şek. 1: Karar Ağacı (derinlik=12) için kategori bazında başarı oranları. Yeşil %80+, sarı %60–80, turuncu %40–60, kırmızı %40 altı başarıyı ifade etmektedir.

Confusion Matrix - Isı Haritası

Tahmin

Gerçek

	BEBEK	DETERJAN T.	ET TAVUK	EV	GIDA	KAĞIT	KOZMETİK	MEYVE SEBZE	SÜT KAHVALT.	SİGARA	İÇECEK
BEBEK				1	7				2		1
DETERJAN T.		21	1	3	3	1	5		2		1
ET TAVUK			43		1				1		1
EV		1		8	3				3		1
GIDA	1	1	3	3	333		2		12		5
KAĞIT		2			4	19	1		3		1
KOZMETİK		3			1	2	13		1		2
MEYVE SEBZE								136			
SÜT KAHVALT.		1	1		7	1	1		113		2
SİGARA			1	1						25	
İÇECEK		3			15		1		7		85

Şek. 2: Karar Ağacı (derinlik=12) için karışıklık matrisi ısı haritası. Koyu mavi hücreler yüksek sıklığı, açık hücreler düşük sıklığı gösterir. Köşegen boyunca yoğunlaşma modelin doğru tahminlerini temsil eder.